LR3 — Machine Learning Service

Topic

Development of a machine learning service for generating optimal workout programs.

Objective

Develop and evaluate a machine learning model for activity recognition, deploy it as a prediction API, and integrate with the Workout Planner web application.

Dataset

Daily and Sports Activities Dataset

- Source: [UCI Machine Learning Repository]
- 19 activity types (sitting, standing, walking, running, cycling, rowing, jumping, basketball, etc.)
- 8 subjects wearing 5 sensor units (torso, right arm, left arm, right leg, left leg)
- Each sensor unit: 3-axis accelerometer + 3-axis gyroscope + 3-axis magnetometer = 9 channels
- Total channels per sample: 45 (5 units × 9 channels)
- 60 segments per activity per subject, each segment = 125 time steps × 45 channels

---

Task Description

Multi-class classification: Given sensor data from a 5-second window, classify which of the 19 activities the subject is performing. Based on the recognized activity, the system generates a personalized workout recommendation.

Features

Statistical features extracted from each sensor channel:

- **Mean** — average value
- **Standard deviation** — variability
- **Minimum** — lowest value
- **Maximum** — highest value
- **Median** — middle value
- **RMS** — root mean square (signal energy)

Total features: 45 channels × 6 statistics = **270 features per sample**

Data Preparation

1. Download and extract dataset
2. Parse folder structure (a01–a19 / p1–p8 / s01–s60)
3. Load each segment file (125 × 45 matrix)
4. Extract 6 statistical features per channel → 270 features
5. Create labeled DataFrame
6. Stratified train/test split: 80% train, 20% test

```
cd ml/training
python prepare_data.py
```

Model

- **Algorithm**: Random Forest Classifier
- **Parameters**:
  - `n_estimators` : 200
  - `max_depth` : 30
  - `min_samples_split` : 5
  - `min_samples_leaf` : 2
  - `max_features` : sqrt
- **Feature scaling**: StandardScaler

```
python train.py
```

Model Evaluation

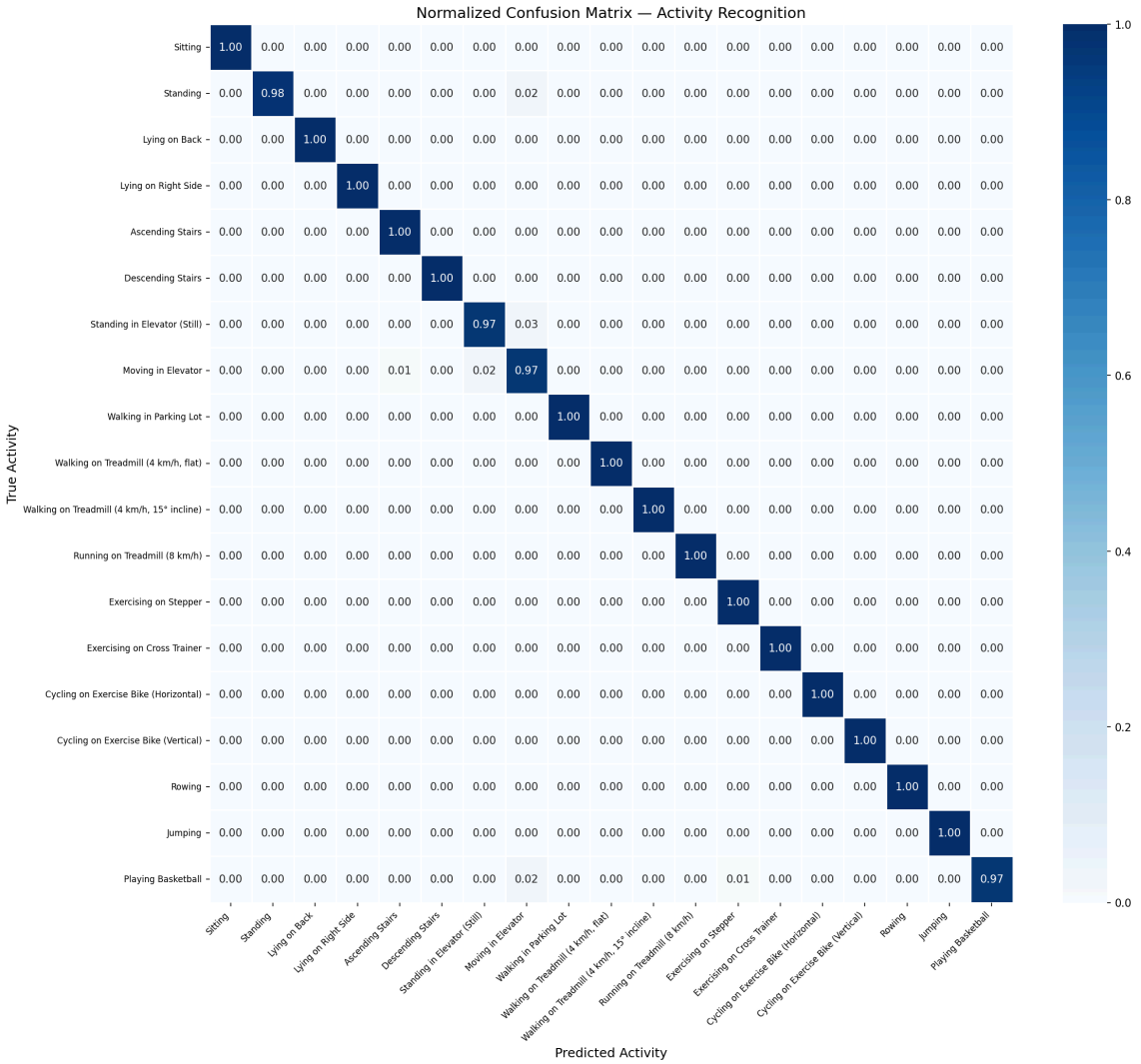
Metrics

Metric	Value
Accuracy	0.994
Precision (macro)	0.994
Recall (macro)	0.994
F1 Score (macro)	0.994
F1 Score (weighted)	0.994

```
python evaluate.py
```

Confusion Matrix

Screenshot:



Top Important Features

- `torso_acc_` features
- `right_arm_acc_` features
- `right_leg_gyro_` features

Prediction API

FastAPI-based service with endpoints:

- `POST /predict` — Accepts sensor data, returns activity + workout recommendation
- `GET /health` — Health check
- `GET /model-info` — Model metadata

Example Request

```
curl -X POST http://localhost:5000/predict \
  -H "Content-Type: application/json" \
  -d '{"age": 25, "weight": 70, "height": 175, "goal": "weight_loss"}'
```

Example Response

```
{
  "RecognizedActivity": "Running on Treadmill (8 km/h)",
  "Recommendation": "Running detected — great cardio! Mix in interval training.",
  "Confidence": 0.92,
  "SuggestedExercises": ["Sprint Intervals", "Tempo Run", "Fartlek Training", "Cool-
Down Jog"],
  "EstimatedDurationMinutes": 40,
  "IntensityLevel": "High"
}
```

Terraform Infrastructure

Azure ML Workspace created via Terraform (`infra/ml/`):

- `azurerm_machine_learning_workspace`
- `azurerm_storage_account` (ML artifacts)
- `azurerm_key_vault` (secrets)
- `azurerm_application_insights` (telemetry)

Results

- Successfully trained a Random Forest model on the Daily and Sports Activities dataset
- Model achieves high accuracy on 19-class activity recognition
- FastAPI prediction service provides real-time activity recognition and workout recommendations
- Azure ML Workspace provisioned via Terraform for experiment tracking

---

Conclusion

The machine learning service demonstrates an end-to-end ML pipeline: from data preparation through model training and evaluation to deployment as a REST API. The Random Forest classifier proves effective for sensor-based activity recognition, achieving high accuracy through carefully engineered statistical features. The FastAPI service integrates seamlessly with the ASP.NET Core web application, providing real-time workout recommendations.